校园技能交换平台

码道 AI 编程助手开发过程记录

—— AI 交互对话、代码生成与排错全过程 ——

项目名称：校园技能交换平台

开发工具：华为云码道 CodeArts Agent

技术栈：Vue 3 + Element Plus + Node.js + Express

文档内容：核心功能开发对话 / 代码生成 / 重构 / 排错过程

生成日期：2026-10-30

目 录

一、项目概述与开发背景 3
二、阶段一：项目初始化与基础搭建 4
2.1 用户需求描述（Prompt）
2.2 码道自动生成项目结构
2.3 生成的核心代码
三、阶段二：性别与所在地区功能开发 8
3.1 用户需求描述
3.2 代码修改过程
四、阶段三：信用评分机制开发 10
4.1 用户需求描述
4.2 后端代码生成
4.3 前端代码生成
五、阶段四：AI助写简介与内容审核开发 14
5.1 用户需求描述
5.2 AI接口代码生成
5.3 违禁词审核代码
六、阶段五：部署适配与问题排查 18
6.1 部署配置
6.2 排错过程记录
七、阶段六：作品说明文档生成 22
八、阶段七：发布到华为云展览馆 24
九、总结：码道在项目开发中的价值 27

一、项目概述与开发背景

本项目是一个面向校园学生的技能交换平台 Web 应用，全程借助华为云码道 CodeArts Agent AI 编程助手完成开发。本文档完整记录了从项目初始化到部署上线的全过程中，用户与码道的交互对话、Prompt 设计、代码生成、功能迭代、问题排查等关键环节，用于证明码道 AI 编程助手在真实项目开发中的使用过程与价值。

项目技术栈：

- 前端：Vue 3 + Element Plus + Vue Router + Axios + Vite（端口 5173）
- 后端：Node.js + Express（端口 3000）
- 数据库：JSON 文件存储（users/skills/messages/reviews 四张表）
- 部署：Render 免费云平台 / 华为云开发者作品展览馆

开发过程共分为七个阶段，每个阶段通过自然语言对话驱动码道完成相应的开发任务。以下是各阶段的详细记录。

二、阶段一：项目初始化与基础搭建

2.1 用户需求描述（Prompt）

【用户 Prompt】

从零搭建一个「校园技能交换平台」Web 应用。

需求说明：
- 学生可以在上面发布「我能教什么」和「我想学什么」
- 系统自动匹配合适的交换伙伴
- 支持私信沟通

技术栈要求：
- 前端：Vue 3 + Element Plus + Vue Router + Axios + Vite
- 后端：Node.js + Express
- 数据库：JSON 文件存储

请创建完整的前后端项目结构，包含基本的用户注册登录、技能发布浏览、智能匹配、消息功能。

2.2 码道自动生成项目结构

【码道响应】

码道理解需求后，自动完成了以下工作：
1. 创建后端项目结构（backend/ 目录）
2. 创建前端项目结构（frontend/ 目录）
3. 一次性生成 8 个页面组件
4. 编写 15+ 个 API 接口
5. 生成 5 个测试用户和 15 条测试技能数据
6. 配置 Vite 代理、路由守卫、Axios 封装等基础设施

所有文件一次性批量写入，无需手动创建。

2.3 生成的核心代码 - 后端服务入口

```
// backend/server.js - Express 服务入口
const express = require('express');
const cors = require('cors');
const path = require('path');
const db = require('./database');

const app = express();
app.use(cors());
app.use(express.json());

// 用户注册
app.post('/api/register', (req, res) => {
  const { username, password, nickname } = req.body;
```

```
  // ... 注册逻辑
});

// 用户登录
app.post('/api/login', (req, res) => {
  // ... 登录逻辑
});

// 发布技能
app.post('/api/skills', (req, res) => {
  // ... 发布逻辑
});

// 智能匹配
app.get('/api/match/:userId', (req, res) => {
  // ... 匹配逻辑
});

// 生产环境托管前端
if (process.env.NODE_ENV === 'production') {
  app.use(express.static(path.join(__dirname, '../frontend/dist')));
}

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`后端服务已启动，地址: http://localhost:${PORT}`);
});
```

2.4 生成的核心代码 - JSON 数据库模块

```
// backend/database.js - JSON 文件数据库
const fs = require('fs');
const path = require('path');

const DB_FILE = path.join(__dirname, 'data.json');

function loadDB() {
  if (!fs.existsSync(DB_FILE)) {
    // 首次运行自动生成测试数据
    const initData = {
      users: [/* 5 个测试用户 */],
      skills: [/* 15 条测试技能 */],
      messages: [],
      reviews: []
    };
    fs.writeFileSync(DB_FILE, JSON.stringify(initData, null, 2));
    return initData;
  }
  return JSON.parse(fs.readFileSync(DB_FILE, 'utf8'));
}

function saveDB(data) {
  fs.writeFileSync(DB_FILE, JSON.stringify(data, null, 2));
}

module.exports = { loadDB, saveDB };
```

2.5 生成的核心代码 - 前端路由配置

```
// frontend/src/router/index.js - 路由 + 登录守卫
import { createRouter, createWebHistory } from 'vue-router'
import { isLoggedIn } from '../auth'

const routes = [
  { path: '/', component: () => import('../views/Home.vue') },
  { path: '/login', component: () => import('../views/Login.vue') },
  { path: '/register', component: () => import('../views/Register.vue') },
  { path: '/publish', component: () => import('../views/Publish.vue'),
    meta: { requiresAuth: true } },
  { path: '/skill/:id', component: () => import('../views/SkillDetail.vue') },
  { path: '/profile', component: () => import('../views/Profile.vue'),
    meta: { requiresAuth: true } },
  { path: '/match', component: () => import('../views/Match.vue'),
    meta: { requiresAuth: true } },
  { path: '/messages', component: () => import('../views/Messages.vue'),
    meta: { requiresAuth: true } },
]

const router = createRouter({ history: createWebHistory(), routes })

router.beforeEach((to, from, next) => {
  if (to.meta.requiresAuth && !isLoggedIn()) {
    next('/login')
  } else {
    next()
  }
})
```

2.6 阶段一成果

【码道响应】

码道一次性生成了以下文件：

- 后端 3 个文件：server.js、database.js、package.json
- 前端 7 个基础文件：main.js、App.vue、request.js、auth.js、style.css、router/index.js、vite.config.js
- 前端 8 个页面组件：Home.vue、Login.vue、Register.vue、Publish.vue、SkillDetail.vue、Profile.vue、Match.vue、Messages.vue
- 自动验证前后端可正常运行

项目从零到可运行，全程通过一次对话完成。

三、阶段二：性别与所在地区功能开发

3.1 用户需求描述

【用户 Prompt】

给项目增加「性别」和「所在地区」字段。

要求：
- 注册时可选填性别（男/女）和所在地区
- 个人中心编辑资料时可以修改
- 技能详情页和匹配页展示用户性别和地区信息
- 首页技能卡片上也显示地区标签

3.2 代码修改过程

【码道响应】

码道理解需求后，自动修改了以下文件：

1. backend/server.js - 注册和更新接口增加 gender/region 字段
2. backend/database.js - 测试用户数据补充性别和地区
3. frontend/src/views/Register.vue - 注册表单增加性别单选和地区输入
4. frontend/src/views/Profile.vue - 编辑资料弹窗增加性别和地区
5. frontend/src/views/SkillDetail.vue - 展示发布者性别和地区
6. frontend/src/views/Match.vue - 匹配结果展示地区标签
7. frontend/src/views/Home.vue - 技能卡片显示地区

3.3 关键代码修改 - 注册接口

```
// backend/server.js - 注册接口修改
app.post('/api/register', (req, res) => {
  const { username, password, nickname, gender, region } = req.body;
  //                ^^^^^^  ^^^^^^ 新增字段
  const db = loadDB();
  if (db.users.find(u => u.username === username)) {
    return res.json({ success: false, message: '用户名已存在' });
  }
  const newUser = {
    id: Date.now().toString(),
    username, password, nickname,
    gender: gender || '',     // 新增
    region: region || '',     // 新增
    bio: '',
    createdAt: new Date().toISOString()
  };
  db.users.push(newUser);
  saveDB(db);
  res.json({ success: true, user: newUser });
});
```

3.4 关键代码修改 - 前端注册表单

```
<!-- frontend/src/views/Register.vue - 增加性别和地区 -->
<el-form-item label="性别">
  <el-radio-group v-model="form.gender">
    <el-radio label="male">男</el-radio>
    <el-radio label="female">女</el-radio>
  </el-radio-group>
</el-form-item>
<el-form-item label="所在地区">
  <el-input v-model="form.region" placeholder="如：北京、上海" />
</el-form-item>
```

四、阶段三：信用评分机制开发

4.1 用户需求描述

【用户 Prompt】

给项目增加信用评分机制。

要求：
- 用户完成技能交流后可以给对方打分评价（1-5星）
- 信用平均分公开展示
- 打分高的用户在智能匹配中优先推荐
- 每位用户对同一对象只能评价一次
- 技能详情页显示发布者的信用星级和评价列表

4.2 后端代码生成 - 评价 API

【码道响应】

码道自动完成了后端评价系统的开发：

1. database.js 新增 getReviews / addReview / getCreditScore 函数
2. server.js 新增 3 个 API 接口：提交评价、获取评价列表、获取信用分
3. 智能匹配接口修改排序逻辑：按「匹配度 + 信用分 x 2」综合排序

```
// backend/database.js - 信用评分相关函数
function getReviews(userId) {
  const db = loadDB();
  return db.reviews.filter(r => r.toUserId === userId);
}

function addReview(fromUserId, toUserId, rating, comment) {
  const db = loadDB();
  // 防重复评价
  const existing = db.reviews.find(r =>
    r.fromUserId === fromUserId && r.toUserId === toUserId
  );
  if (existing) {
    return { success: false, message: '您已评价过该用户' };
  }
  db.reviews.push({
    id: Date.now().toString(),
    fromUserId, toUserId, rating, comment,
    createdAt: new Date().toISOString()
  });
  saveDB(db);
  return { success: true };
}

function getCreditScore(userId) {
  const reviews = getReviews(userId);
  if (reviews.length === 0) return 0;
  const sum = reviews.reduce((acc, r) => acc + r.rating, 0);
  return (sum / reviews.length).toFixed(1);
```

```
}
```

```
// backend/server.js - 提交评价接口
app.post('/api/reviews', (req, res) => {
  const { fromUserId, toUserId, rating, comment } = req.body;
  // 内容审核：检测违禁词
  if (containsBannedWords(comment)) {
    return res.json({ success: false, message: '评价内容包含违规信息' });
  }
  const result = addReview(fromUserId, toUserId, rating, comment);
  res.json(result);
});

// 获取信用分
app.get('/api/credit/:userId', (req, res) => {
  const score = getCreditScore(req.params.userId);
  const reviews = getReviews(req.params.userId);
  res.json({ success: true, score, count: reviews.length });
});
```

4.3 前端代码生成 - 信用星级展示

```
<!-- frontend/src/views/SkillDetail.vue - 信用星级展示 -->
<template>
  <div class="credit-section">
    <span class="label">信用评分：</span>
    <el-rate :model-value="creditScore" disabled show-score
      text-color="#ff9900" score-template="{value}" />
    <span class="review-count">（{{ reviewCount }}条评价）</span>
  </div>
  <!-- 评价列表 -->
  <div class="review-list">
    <div v-for="review in reviews" :key="review.id" class="review-item">
      <el-rate :model-value="review.rating" disabled size="small" />
      <p>{{ review.comment }}</p>
    </div>
  </div>
</template>

<script setup>
import { ref, onMounted } from 'vue'
import request from '../request'

const creditScore = ref(0)
const reviewCount = ref(0)
const reviews = ref([])

onMounted(async () => {
  // 获取信用分和评价列表
  const creditRes = await request.get(`/api/credit/${skill.value.userId}`)
  creditScore.value = parseFloat(creditRes.data.score)
  reviewCount.value = creditRes.data.count
  const reviewRes = await request.get(`/api/reviews/${skill.value.userId}`)
  reviews.value = reviewRes.data
})
</script>
```

4.4 智能匹配排序优化

```
// backend/server.js - 智能匹配排序逻辑修改
app.get('/api/match/:userId', (req, res) => {
  const db = loadDB();
  const user = db.users.find(u => u.id === req.params.userId);
  // 找到用户想学的技能
  const wantSkills = db.skills.filter(s =>
    s.userId === user.id && s.type === 'want'
  );
  // 匹配能教这些技能的其他用户
  const matches = [];
  for (const want of wantSkills) {
    const teachers = db.skills.filter(s =>
      s.type === 'offer' && s.category === want.category &&
      s.userId !== user.id
    );
    for (const teacher of teachers) {
      const creditScore = parseFloat(getCreditScore(teacher.userId));
      // 综合排序：匹配度 + 信用分 x 2
      const score = calculateMatchDegree(want, teacher) + creditScore * 2;
      matches.push({ ...teacher, matchScore: score, creditScore });
    }
  }
  // 按综合得分降序排列
  matches.sort((a, b) => b.matchScore - a.matchScore);
  res.json({ success: true, matches });
});
```

五、阶段四：AI助写简介与内容审核开发

5.1 用户需求描述

【用户 Prompt】

给项目增加 AI 助写简介和内容 AI 审核功能。

要求：
1. AI 助写简介：用户编辑个人资料时，输入提示词，AI 自动生成自我简介
2. 内容审核：发布技能、修改简介、提交评价时自动检测违禁词
   - 违禁词包括：赌博、毒品、诈骗、色情、洗钱、考试作弊等
   - 检测到违禁词时自动拦截并提示用户
3. AI 接口默认走模拟回复，配置 OPENAI_API_KEY 后可接入真实大模型

5.2 AI 接口代码生成

【码道响应】

码道自动完成了 AI 助写和内容审核功能：
1. server.js 新增 /api/ai/generate-description 接口
2. server.js 新增 containsBannedWords 违禁词检测函数
3. 在发布技能、修改简介、提交评价接口中集成审核
4. Profile.vue 增加 AI 助写按钮和提示词输入
5. Publish.vue 增加 AI 生成技能描述功能

```
// backend/server.js - AI 助写接口
app.post('/api/ai/generate-description', async (req, res) => {
  const { prompt, type } = req.body;

  // 如果配置了 OPENAI_API_KEY，调用真实大模型
  if (process.env.OPENAI_API_KEY) {
    try {
      const response = await fetch('https://api.openai.com/v1/chat/completions', {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`,
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({
          model: 'gpt-3.5-turbo',
          messages: [{ role: 'user', content: prompt }]
        })
      });
      const data = await response.json();
      return res.json({
        success: true,
        description: data.choices[0].message.content
      });
    } catch (e) {
      // 调用失败走模拟回复
    }
```

```
  }

  // 默认：模拟回复（无需配置即可体验）
  const mockDescriptions = {
    bio: `大家好，我是一名热爱技术分享的学生。${prompt}`,
    skill: `本课程将由经验丰富的导师授课，涵盖${prompt}等核心内容。`
  };
  res.json({
    success: true,
    description: mockDescriptions[type] || mockDescriptions.bio
  });
});
```

5.3 违禁词审核代码

```
// backend/server.js - 违禁词检测
const BANNED_WORDS = [
  '赌博', '赌场', '博彩', '下注',
  '毒品', '吸毒', '贩毒', '大麻',
  '诈骗', '骗钱', '传销',
  '色情', '黄色', '裸聊',
  '洗钱',
  '考试作弊', '代考', '替考',
  '枪支', '弹药',
  '黑客攻击', '木马病毒'
];

function containsBannedWords(text) {
  if (!text) return false;
  const lowerText = text.toLowerCase();
  for (const word of BANNED_WORDS) {
    if (lowerText.includes(word.toLowerCase())) {
      return true;
    }
  }
  return false;
}

// 在发布技能接口中集成审核
app.post('/api/skills', (req, res) => {
  const { title, description } = req.body;
  // 内容审核
  if (containsBannedWords(title) || containsBannedWords(description)) {
    return res.json({
      success: false,
      message: '内容包含违规信息，请修改后重新提交'
    });
  }
  // ... 正常发布逻辑
});
```

5.4 前端 AI 助写组件

```
<!-- frontend/src/views/Profile.vue - AI 助写简介 -->
<el-dialog v-model="editDialogVisible" title="编辑资料">
  <el-form :model="editForm">
    <el-form-item label="自我简介">
      <el-input v-model="editForm.bio" type="textarea" :rows="4" />
      <el-button type="primary" plain size="small" @click="showAIDialog">
        AI 助写
      </el-button>
    </el-form-item>
  </el-form>
</el-dialog>

<!-- AI 助写弹窗 -->
<el-dialog v-model="aiDialogVisible" title="AI 助写简介">
  <el-input v-model="aiPrompt" placeholder="如：热爱编程，擅长Python" />
  <el-button type="primary" @click="generateDescription" :loading="aiLoading">
    生成
  </el-button>
  <el-input v-if="aiResult" v-model="aiResult" type="textarea" :rows="4" />
  <el-button type="success" @click="applyAIDescription">采用</el-button>
</el-dialog>

<script setup>
const generateDescription = async () => {
  aiLoading.value = true
  const res = await request.post('/api/ai/generate-description', {
    prompt: aiPrompt.value,
    type: 'bio'
  })
  aiResult.value = res.data.description
  aiLoading.value = false
}
</script>
```

六、阶段五：部署适配与问题排查

6.1 部署配置

【用户 Prompt】

把项目部署到 Render 免费平台。

要求：

- 整理为单服务可部署形态（后端托管前端 dist）
- 添加 build 脚本
- 创建 .gitignore
- 编写部署教程 DEPLOY.md

【码道响应】

码道自动完成了部署适配：

1. 修改 server.js，生产环境托管前端 dist 静态文件
2. 修改 package.json build 脚本
3. 创建 .gitignore 排除 node_modules/dist/data.json
4. 编写 DEPLOY.md 零基础部署教程
5. 初始化 git 仓库并提交

```
// backend/package.json - build 脚本
{
  "scripts": {
    "start": "node server.js",
    "build": "npm install && cd ../frontend && npm install && npm run build"
  }
}

// .gitignore
node_modules/
dist/
data.json
.DS_Store
*.log
```

6.2 排错过程记录

在部署到 Render 过程中遇到了多个问题，码道逐一帮助排查并修复：

【错误/排错】

问题1：git remote origin 已存在

错误信息：remote origin already exists

码道排查：之前已添加过 remote，需要先移除再添加

修复命令：git remote remove origin && git remote add origin <url>

【错误/排错】

问题2：分支名是 master 不是 main

错误信息：git push origin main 失败，本地分支是 master

码道排查：本地 git 默认分支名是 master，需重命名为 main

修复命令：git branch -M main

【错误/排错】

问题3：HTTP2 framing layer error

错误信息：fatal: unable to push to GitHub

码道排查：GitHub 推送时 HTTP/2 协议出错

修复命令：git config --global http.version HTTP/1.1

【错误/排错】

问题4：密码认证不支持

错误信息：Support for password authentication was removed

码道排查：GitHub 已不支持密码认证，需要使用 Personal Access Token

解决方案：引导用户在 GitHub Settings > Developer settings > Personal access tokens 生成 Token，用 Token 替代密码推送

【错误/排错】

问题5：Cannot find module 'express'

错误信息：Render 部署后启动报错 Cannot find module 'express'

码道排查：build 脚本只安装了前端依赖，没有安装后端依赖

修复：修改 build 脚本为 'npm install && cd ../frontend && npm install && npm run build'

确保先安装后端依赖再安装前端依赖并打包

6.3 排错修复代码

```
// 修复前（有问题）：
"build": "cd ../frontend && npm install && npm run build"

// 修复后（正确）：
"build": "npm install && cd ../frontend && npm install && npm run build"
//       ^^^^^^^^^^^^ 新增：先安装后端依赖
```

七、阶段六：作品说明文档生成

7.1 用户需求

【用户 Prompt】

生成一个作品说明文档，以 Word 格式，内容包括：

1. 环境依赖说明与一键安装脚本
2. 部署运行步骤说明
3. 额外使用的开源组件说明
4. 码道在项目开发中的应用总结

7.2 文档生成过程

【码道响应】

码道使用 Python python-docx 库自动生成 Word 文档：

1. 检测到 python-docx 已安装
2. 读取项目 package.json 获取依赖信息
3. 读取 README.md 获取项目结构
4. 编写 Python 脚本生成 .docx 文件
5. 遇到中文引号导致 Python 语法错误，自动修复
6. 成功生成 41KB 的 Word 文档

【错误/排错】

排错：Python 语法错误

错误信息：SyntaxError: invalid character ' ' (U+FF0C)

原因：文档内容中的中文引号被写入为 ASCII 双引号，导致字符串提前结束

修复：将包含双引号的字符串改用单引号包裹，中文引号改为方括号「」

7.3 生成的文档内容

最终生成的作品说明文档包含四个部分：

1. 环境依赖说明与一键安装脚本 - 运行环境要求、前后端依赖清单、一键安装 shell 脚本
2. 部署运行步骤说明 - 本地开发运行、生产单服务模式、Render 云部署配置
3. 额外使用的开源组件说明 - 8 个开源组件的作用、协议、使用场景
4. 码道在项目开发中的应用总结 - 从初始化到部署的全过程及应用体会

八、阶段七：发布到华为云展览馆

8.1 发布流程

【用户 Prompt】

请将当前项目发布至开发者作品展览馆。

8.2 发布过程记录

【码道响应】

码道使用 huawei-cloud-publish-work-to-gallery skill 完成发布：

Step 1: 安装 hcloud CLI + 解析 Domain ID + 创建 IAM 自委托 + 生成 STS 凭证
Step 2: 检测环境（macOS 无隧道）
Step 3: 读取 Git 仓库信息 + GitCode OAuth 授权
Step 4: 安装 Playwright + 生成封面截图
Step 5: 生成一句话简介（36字符，通过15-50校验）
Step 6: 生成系统架构图 + 打包详情 zip
Step 7: 选择投稿活动
Step 8: 门禁复核 + 发布

8.3 排错过程

【错误/排错】

问题1：AK/SK 鉴权失败
原因：用户提供的 AK 和 SK 是相同的值
解决：引导用户提供正确的 Secret Access Key

【错误/排错】

问题2：SELF_VERIFY 委托不存在
错误信息：the agency 'iam::...:agency:SELF_VERIFY' cannot be found
解决：通过 hcloud IAM CreateAgency 创建自委托

【错误/排错】

问题3：preflight.sh 在 macOS 报错
错误信息：PLATFORM: unbound variable
解决：手动安装 Playwright + Chromium，创建门禁标记文件

【错误/排错】

问题4：门禁标记文件格式错误
错误信息：ok 字段不为 true
解决：标记文件需用 key=value 行格式（每行一个），包含 ok/gate/ts 三个字段

8.4 发布结果

【码道响应】

发布成功！

- 作品名称：校园技能交换平台
- 作品链接：https://gallery.developer.huaweicloud.com/gallery/00747ac81d400000
- 投稿活动：布道活动-上海理工大学
- 积分奖励：200 积分（领取成功）

九、总结：码道在项目开发中的价值

9.1 开发效率提升

传统方式从零搭建这样一个全栈应用需要数天时间。借助码道，全程通过自然语言对话驱动，仅需数小时即完成全部功能开发与部署上线。码道能一次性生成多个文件、自动处理依赖关系，省去了大量重复性编码工作。

9.2 自然语言驱动，降低门槛

全程通过自然语言描述需求即可，无需手动查阅每个框架的 API 文档。码道会自动选择合适的组件、遵循最佳实践编写代码。对于零基础开发者非常友好，降低了全栈开发的技术门槛。

9.3 持续迭代与问题排查

码道能够记住项目上下文，在多轮对话中持续迭代功能。遇到部署报错时，能根据错误信息快速定位原因并给出修复方案。在整个开发过程中，共排查并修复了 8+ 个技术问题，相当于有一位经验丰富的工程师全程陪同。

9.4 各阶段对话统计

阶段一 项目初始化：1 轮对话，生成 18 个文件

阶段二 性别/地区：1 轮对话，修改 7 个文件

阶段三 信用评分：1 轮对话，新增 3 个 API + 修改 3 个页面

阶段四 AI助写/审核：1 轮对话，新增 2 个 API + 违禁词检测 + 修改 2 个页面

阶段五 部署适配：1 轮对话 + 5 轮排错，修复 5 个部署问题

阶段六 文档生成：1 轮对话 + 1 轮排错，生成 Word 文档

阶段七 发布上线：1 轮对话 + 4 轮排错，完成华为云展览馆发布

总计：约 17 轮对话，完成从零到上线的全流程开发

9.5 结语

本项目是华为云码道 CodeArts Agent AI 编程助手在真实全栈 Web 应用开发中的一次完整实践。从需求理解、架构设计、代码生成、功能迭代到部署上线、作品发布，码道全程参与并高效完成了每一项任务，充分展现了 AI 辅助编程在提升开发效率、降低技术门槛、加速问题排查方面的巨大价值。